

Cornserve

Summary of Cornserve : Efficiently Serving Any to Any Models

Anika Misra (anmisra), Joshua Nigrelli (jnigrell), Nandana Sheri (nansheri), Namita Shah (namishah)

Problem and Motivation

The popularity of multimodal models has surged throughout recent years. However, most existing systems either focus only on text-only models, or they may focus on multimodal LLMs, which still output only text. Any-to-any models, on the other hand, are models that take any modality as input and any modality as output (mostly text, image, video, and audio). Because most existing systems focus on a single modality, there is still no online serving system for any-to-any models. Even when not focused on text-only models, other prior works still focus on singular modality models like image or video. Because any-to-any models consist of many different components and modalities, creating one system for them is a challenge.

The authors mention that the model deployment strategy for any-to-any models is an instance of the multicommodity network design problem (MCNDP). This is because there are multiple request types (image-to-text, text-to-video, audio-to-text, etc.) with fixed GPU costs and constraints. Optimizing total workload resource usage with these fixed constraints then becomes an NP-hard problem in the general case.

When specifically working with Any-to-Any Models, each inference request that the system receives contains different combinations of input modalities which creates different use cases for image encoders, LLMs and generators. This creates a request and computation path heterogeneity which is an NP Hard Network Design Problem. Another type of heterogeneity is computation scaling heterogeneity, caused by different components of the any-to-any models (encoders, LLMs, generators) running at different speeds. Existing solutions implement these components through collocation on a single monolithic server that can bottleneck the throughput. Current systems enforce either keeping everything on one server or splitting it all apart, offering no middle ground to mix both setups for the same workload. Thus, the best deployment strategy in such a case would be heavily dependent on the workload of the model and the model itself and the implementation of an automated planner that can pick the best of all worlds.

Related Works

LLM Serving Systems and Optimizations: Many prior works on LLM serving only consider text-only LLMs, or multimodal LLMs that still only output text. The disaggregation strategies they propose include Prefill-Decode, Encode-Prefill-Decode, or other disaggregation techniques based on the modality alone. On the other hand, for model-level optimization, existing systems extend Kubernetes (an automatic scaling and deployment system) or create optimized, blackbox containers. Regardless, all of these methods are still specifically only for LLMs or unimodal models. Each method promotes only one specific deployment strategy and does not deal with the heterogeneity of any-to-any models.

Multimodal Model Inference: Previous methods regarding image or audio models are designed for a single component. These systems are effective, however, they cannot necessarily work with any-to-any models, which consist of various components. That being said, their individual contributions can still be utilized by Cornserve. There is a need to link together these components to create inference for any-to-any models.

Resource Allocation and Parallelization for Training: The challenge of splitting up an any-to-any model for inference is an analogue to large model parallelization during the training phase. Earlier systems required manually designed strategies, similar to current inference systems that use phase-wise or component-wise disaggregation. However, over time, automated planners emerged, and this is similar to Cornserve's contribution.

Solution Overview

The main aim of Cornserve is a serving system for Any-to-Any Models that optimizes total resource usage (number of GPUs) for a given workload and request rate. Cornserve comprises of several different components highlighted below:

Planner: The planner performs the actual resource allocation of GPUs. It comprises another component called the Profiler that measures throughput for each component of the model under the workload. The planner takes in the valid components and invocation paths as a model executor graph along with a request dataset that is representative of the workload distribution. The goal here is to find the right mix of executor deployment options and number of replicas for each option.

This resource allocation problem as described above is a multicommodity network design problem, which has so many different possibilities and combinations which makes it NP Hard,

making every single GPU a decision variable. (For example, what component should run on each GPU? How do they communicate with each other?). To fix this, the authors here use Cell Based Planning, which places groups of GPUs into 'cells' to reduce the search space and simplify the problem.

Gateway: This is an entry point for control signals such as model deployment, scaling as well as incoming inference requests.

Task Dispatcher: Dispatches model component invocations to the appropriate executors.

Task Manager: Manages a set of Executor replicas for a single model's deployment option.

Task Executor: Executes model components like the encoder, LLM or generator on the GPUs.

When a new model is created for deployment, it is first sent to the Gateway which invokes the Planner to allocate the correct number of resources. Prior to allocation, we need to know what performance and throughput would look like under these workloads so the Profiler is called. Once performance is determined, a deployment strategy is decided. The planner then creates Task Managers which consecutively create Task Executor replicas. Thus, when an inference request is incoming, the Gateway invokes the top level task of the model which gets sent to the Dispatcher and Task Manager. Routing and load balancing are all decided through offline planning such that the load is in adherence to the Planner's decision. As the system runs, the Planner can re-plan by allocation of resources again.

Something unique to Cornserve over a lot of dynamic planning and load balancing solutions is the fact that Cornserve's planner allocates resources considering workload distributions *offline*. Model Components like encoders and LLMs are represented as a library of unit task classes that can be used by model developers to express computation paths for the planner. Routing is also an important part of deployment. The main question here is how to find the right sequence of executors so that they are load balanced. Most strategies in this case are dynamic load balancing like checking the queue of each component. Cornserve takes a different approach with zero dynamic control overhead because the Planner already splits the arrival rate for each request type, this distributes multiple requests across different executor paths in specific ratios. For example, for an image and text request type, the incoming request rate is assigned to the dedicated encoder and monolithic executor in a 7:3 ratio and then requests to the encoder are further split in 5:2 ratio between the monolithic executor and LLM executor.

Limitations

Cornserve's planner works offline for a sample workload distribution that is decided based on request rates from historical data. Though for deviance, it does do replan there is a certain level of overhead that can come from the replan if the workload tends to be bursty and unpredictable. Thus, if the workload is significantly different from the offline planning, static

routing would not necessarily be a good solution here. Cornserve also only primarily looks into request types and disaggregates them according to different executors. However, there are various different points of contention like input level, difficulty of various inference requests that Cornserve does not take into consideration. Even similar request types could have a disproportionate workload which needs to be considered at a more granular level. Another interesting point to note is that the paper does not deeply explore batching of inference requests. To what extent can batching help with inference, whether Cornserve lacks in that aspect and how it can be made better are all points to think about.

Future Research Directions

One future research direction is to have a guarantee for latency in the 99th percentile rather than only the average. However, this is difficult to do due to request sizes in any-to-any models varying so greatly. Another direction of future work is to evaluate Cornserve on dynamic workloads rather than a stable distribution of requests. Third, Cornserve could be scaled towards larger clusters, perhaps through incremental reconfiguration to save time. Finally, memory is important to account for in planners, especially in decode-heavy workloads, and so Cornserve could also potentially incorporate some memory-aware optimizations.

Summary of Class Discussion

- 1. If an AI agent is running in a loop, such as an AI agent thinking, acting, etc., can Cornserve handle that using its one-time record pass?**
 - a. Not really. Cornserve has a static computation graph, but AI agents require dynamic control flow. Hence, it may not work very well in an agentic setup. A dynamic flow would cause a lot of computing overhead, but Cornserve's methodology still works on traditional, non-dynamic, multimodal tasks.
- 2. How exactly does Cornserve split the workload across different server replicas?**
 - a. This is not talked about very precisely. The paper explains how it routes data between components, but the actual math behind the task dispatcher is less clear. The class hypothesized that it uses a "round robin" method.
- 3. If Cornserve groups resources into blackbox cells, how does that affect predicting the latency?**
 - a. Treating a cell like a black box may ignore the internal communication overhead between the GPUs inside it. There is also a potential for resource fragmentation. For example, if there are 2 cells of size 4 each, and a workload requires exactly 5

GPUs to run efficiently, it must give you two full cells (8 GPUs), leaving 3 GPUs sitting idle.

Approximate Caching

Summary of Approximate Caching for Efficiently Serving Text-to-Image Diffusion Models - Agarwal et al (NSDI '24)

Problem and Motivation

Diffusion models (DMs) have become a popular method for the task of text-to-image generation and are widely adopted by companies that offer image generation as a service. DMs generate images starting with random noise, which is then iteratively denoised, conditioned on the input text, to produce an output image matching the prompt. These denoising steps, called diffusion steps, may be performed 50-100 times, and as many as 1000. This process is slow and resource-intensive, increasing latency and limiting the practicality of using DMs in an interactive setting. Reducing the size of the diffusion model may decrease this latency, but at the cost of a significant performance hit. Another potential optimization, using more powerful GPUs when serving, can be prohibitively expensive. This motivates the authors' introduction of their novel text-to-image generation system, NIRVANA.

Related Works

ML optimizations: Prior work improves diffusion model efficiency through techniques such as model distillation, pruning, quantization, but it often comes at the cost of degraded image quality. Systems like DeepSeed optimize distributed inference, but these approaches accelerate the base model rather than reducing the denoising process that NIRVANA targets.

Model-serving in Cloud: Past research in ML serving systems focuses on reducing inference latency via techniques like layer caching, ensembling, autoscaling, and straggler mitigation (i.e. Clipper and Cocktail). These systems optimize deployment efficiency and resource management but they do not change how the diffusion model itself computes an image. In contrast, NIRVANA modifies the inference computation by skipping early denoising steps and reusing cached intermediate noise states.

Text-to-image models: Diffusion models are a dominant class of text-to-image systems, popularized by models such as DALL-E, Imagen, and Stable Diffusion, and widely adopted by

industry. Prior work has focused on accelerating diffusion through faster or parallel sampling methods, but these approaches trade compute for speed and operate independently of NIRVANA, which can be applied on top of any diffusion model.

Caching: Prior work on caching in DNN inference focuses on storing intermediate layer outputs to avoid recomputation, but this differs from NIRVANA's approach of caching intermediate diffusion noise states rather than model activations. Existing cache policies (i.e. LRU, static caching) and retrieval-based image caching methods either do not apply to diffusion workflows or suffer from quality degradation, motivating a specialized caching design.

Approximations in System Design: Prior systems research has used approximation techniques to reduce redundant computation in domains such as big data analytics, mobile computing, and video processing typically by skipping work, reusing partial results, or tolerating bounded inaccuracy to improve efficiency. However, these techniques are not directly applicable to diffusion models because diffusion inference follows a tightly coupled, iterative denoising process where intermediate states evolve sequentially and cannot be trivially skipped or reused without careful reconditioning.

Solution Overview

NIRVANA is based on the technique of approximate caching, which the authors introduce to reduce both the latency and cost associated with serving diffusion models. The standard diffusion process starts with Gaussian noise at state x_N , which is followed by N diffusion steps conditioned on the input prompt. In state x_0 , the model has produced the final image. The approximate caching process skips the first K steps, using the noise at state x_K from a different prompt. The cached noise was conditioned for K steps on a different prompt, and the input prompt is then used to condition the for the final $N-K$ steps. The alternative prompt and the value of K are determined by the similarity of cached prompts to the input prompt. More available similar prompts mean the value of K is larger, meaning that more steps can be skipped and leading to lower latency and cost per prompt. If no suitable prompt is cached, the model runs for the full N steps.

The number of steps skipped, K , is computed by balancing the hit rate and compute savings. The hit rate is the likelihood that an input prompt matches a prompt that the system has already encountered and cached. If it is acceptable for K to be low and fewer steps to be skipped, the hit rate can be made high, as noise taken from early diffusion stages can be applied to produce a larger variety of high-quality final images. If K is higher, there are fewer images

that can be created by denoising the already-partially-diffused image. Thus, the hit rate is lower for higher K . K can be computed by NIRVANA on a case-by-case basis for each prompt, allowing for a balance of savings and quality.

Various characteristics of an image affect the effective value of K for the noise retrieved for a given prompt. Layout, color, shape, objects, and style are found to be determined within a certain percentage of diffusion steps. Retrieved images that vary in some of these characteristics from the prompt can only contribute noise from a certain point of the diffusion process as a result.

The NIRVANA system begins execution by generating an embedding vector for an input prompt using an embedding generator. Optionally, a match predictor module, based on a classifier, predicts if there is a suitable match for the prompt’s embedding in the vector database for the selection of an intermediate state. If a match is likely, the system searches for a suitable intermediate state by first finding the most similar cached embedding to the input embedding, then, using the cache selector module’s heuristics, a suitable intermediate state is selected from the cached embedding’s generation. Not all intermediate states of the cached embedding are stored to balance memory usage with generation quality and latency. The intermediate state is retrieved from NIRVANA’s file system, then passed to the diffusion model to be reconditioned on the prompt for the remaining generation steps. If the match predictor does not predict a likely match, or the closest embedding is not sufficiently similar to the prompt embedding, generation falls back to the vanilla diffusion model.

NIRVANA’s design relies heavily on its cache, designed to retrieve the optimal intermediate state for a given prompt, if such a state exists. Embeddings are stored in a vector database for fast similarity search and intermediate noise states are stored on AWS Elastic File System, which was chosen for its cost and performance balance. The cache contains a cache selector component, which uses a heuristic to determine K based on the characteristics of the input and cached embedding. This heuristic can be tuned to balance compute savings and generation performance.

NIRVANA also introduces a novel cache management policy for the approximate caching system: Least Computationally Beneficial and Frequently Used (LCBFU). This policy is based on the idea that some intermediate states will allow for more computation and latency to be saved than others based on their highest K values and similarity to common prompts. LCBFU assigns intermediate state i an LCBFU score, $f_i * K_i$, where f_i is the access frequency of the item and K_i denotes the diffusion step that i belongs to. Items with the lowest LCBFU scores are evicted from the cache. Insertion in the LCBFU policy involves inserting all intermediate states generated from a cache miss, evicting low-LCBFU-score states when there is no remaining storage. States to be evicted are maintained by a K -min heap that tracks the lowest LCBFU scores. The LCBFU policy can lead to some, not all, states being stored for a particular prompt, with the most beneficial ones remaining in the cache. LCBFU is found in the authors’

experiments to have a hit rate on par with traditional caching techniques while offering a reduction in the amount of computation used.

The authors find that NIRVANA maintains similar image quality to solutions that involve the full diffusion process, with 21% savings in GPU computation, a 19.8% latency reduction, and 19% amortized savings on dollars spent on the generation system. Latency variance is minimized as well, which is beneficial in an interactive deployment. Generation quality measured on quantitative metrics significantly improves compared to retrieval-based baselines, which simply retrieve the most similar image for a prompt rather than generating a new one. Approximate caching is also found not to affect performance while allowing for computational savings. In a test deployment involving 60 users and 1000 images, 79% of users liked the images generated by NIRVANA, in comparison to only 31% that liked the images generated by the GPT-CACHE baseline. A vanilla model produced images that 86% of users liked, showing that there was still some reduction in image quality to be balanced with the cost savings.

Limitations

Longer prompts differ more than shorter prompts with comparable similarity scores. CLIP embeddings, therefore, fail to capture the differences between longer prompts. Such failed estimates in similarity can lead to values of K that are too high to effectively recondition the cached noise on a different input prompt and as a result generation on longer prompts suffers a hit in quality.

Over time, the usage of cached states can limit image diversity as conditioning on noise generated for the same prompts eliminates some of the randomness of the diffusion process. While changing the random seed helps some, the only real way the authors suggest to fix this is by generating some images from scratch with the vanilla diffusion method, which eliminates the savings NIRVANA allows for.

If the characteristics of the prompts the system sees change over time, there will be fewer similar items available for retrieval in the cache. In this case, K may be lowered in many queries and the match predictor may force vanilla generation. Quality won't necessarily suffer, by design, but computational savings will again be limited.

The LCBFU cache management policy prioritizes compute savings rather than quality. The eviction of cached items with lower K , encouraged by the $f_i * K_i$ formula, removes cached items that may be more frequently used. Again, this decision was based on compute savings but may lead to less diversity in images and slightly lower image quality.

Potential future generative diffusion models that use fewer denoising steps will decrease NIRVANA's efficacy as the diffusion process that NIRVANA optimizes will inherently be less computationally expensive for the vanilla model. Eventually the cache and classification

overheads will outweigh generation time, with the added bonus of generations that were never conditioned on a different prompt.

Future Research Directions

Multiple limitations of the NIRVANA system relate to the caching system's limited ability to retain diversity in the images it generates. Future work could attempt to overcome this limitation, allowing for more diversity in image generation while maintaining the compute savings such a system offers.

Diffusion models are used in multiple other modalities, such as video and audio. Recently, diffusion-based language models have been introduced as well. Future research could focus on a similar noise caching method for prompts in these domains.

Many related works focus on scheduling optimizations in model deployments. In a large-scale deployment using NIRVANA, some of these optimizations could be implemented to potentially further reduce latency for an end user.

Based on class discussion, measures to implement a security layer for NIRVANA would protect against cache poisoning, malicious prompt injection, and unintended cross-user influence. For example, one approach would be to introduce cache isolation policies, such as per-user partitions, so that intermediate diffusion states are not globally shared without safeguards. The system could also monitor unusual access patterns or similarity anomalies in the vector database, flagging potential poisoning attempts.

Summary of Class Discussion

1. Why is this idea booming now? Couldn't others have done this earlier?

- a. The key novelty lies in demonstrating approximate caching at production scale. Prior to large deployments like Adobe Firefly, there wasn't sufficient prompt volume to analyze reuse patterns or evaluate whether caching intermediate diffusion states would meaningfully reduce computation. This is arguably the first work to validate such a caching strategy on large-scale, real-world workloads.

2. Is caching the same thing as KV caching in Transformers?

- a. Not really. KV caching stores intermediate key-value states within a single sequence to accelerate autoregressive token generation. NIRVANA caches intermediate diffusion states across different prompts and even across users. KV cache reuses work within one request; NIRVANA reuses work across requests, more like mooncake with global cache across users (abstract level).

3. **How does NIRVANA handle a drastic change in prompt topics? Is there a cost for a more up-to-date predictor?**
 - a. If the prompt distribution shifts significantly, the system retrains and updates the match predictor based on the updated vector database. The cache adapts dynamically as new prompts are inserted and older, less useful ones are evicted.
4. **If CLIP embeddings are weak for long prompts, why not use a better encoder?**
 - a. At the time, CLIP was compared primarily against BERT and performed SOTA for alignment with diffusion conditioning. However, this is a known limitation - long-context prompts degrade similarity detection. Stronger encoders like LLM-based semantic encoders could improve similarity, but the deeper issue is alignment with diffusion model semantics. Even if two prompts are semantically similar, the diffusion model must still be able to recondition the intermediate noise effectively.
5. **Compared to other caching performance, NIRVANA performs the best but it's still recreating the whole image from cache. Is there a way to implement quality measures? How good would the image be? Some heuristic?**
 - a. The current policy optimizes hit rate and number of steps saved (compute benefit). Full regeneration always produces the best quality but sacrifices savings. Incorporating quality is difficult because evaluating quality requires generating the image adding overhead.
 - b. Potential heuristics (i.e. FID-like approximations) could be explored, but real-time quality prediction remains challenging.
6. **Would caching reduce diversity for the same prompt?**
 - a. Potentially yes. Reusing intermediate states may reduce randomness compared to starting from fresh noise. The authors analyze which concepts “freeze” at certain diffusion steps and sometimes evict or limit reuse to maintain some diversity. Professor mentioned they could limit reuse frequency (i.e. discard after “X” uses) to inject randomness. Their cache size was 1.5 TB.
7. **Would we need a security layer added to this? Could malicious prompts or images pollute the cache?**
 - a. This remains an open issue; adversarial injection into shared caches is not taken care of in this paper.
8. **Why is this paper titled CornServe?**
 - a. It derives from another paper who used “cornstarch” which also relates to multimodal serving. It's also a distinctive name that improves discoverability in research.
9. **Would this caching approach work for language diffusion models?**

- a. Possibly yes. If a language model follows an iterative denoising paradigm (noise $\rightarrow$ condition $\rightarrow$ refine), intermediate-state reuse could theoretically apply. The key requirement is a structured, stepwise refinement process similar to image diffusion.